

Ubuntu Server Automation and Migration using Bash Scripts

1.1 Introduction

As organizations grow, manually configuring and maintaining servers becomes increasingly time-consuming, repetitive, and prone to human error. Tasks such as installing software packages, configuring secure remote access, synchronizing server data, and migrating applications often require administrators to execute numerous commands manually on each server. This approach is inefficient and can lead to inconsistent server configurations.

To address these challenges, this project implements a Bash-based automation solution for Ubuntu Linux servers. The automation uses Bash scripting to streamline server deployment, configuration, synchronization, and migration tasks between two Ubuntu virtual machines.

The project was developed using two Ubuntu virtual machines:

- Ubuntu01 – Source Server
- Ubuntu02 – Destination Server

The implementation automates several administrative tasks, including:

- Initial server configuration
- Installation of required software packages
- SSH configuration
- Passwordless SSH authentication
- Server file synchronization
- Installed package migration
- Nginx load balancing configuration

By automating these tasks, the deployment process becomes faster, more reliable, and easier to reproduce whenever a new Ubuntu server is introduced into the environment.

1.2 Aim of the Project

The aim of this project is to automate the deployment, configuration, synchronization, and migration of Ubuntu Linux servers using Bash scripts in order to reduce manual administration and improve consistency across multiple servers.

1.3 Project Objectives

The objectives of this project are to:

- Automate the initial configuration of Ubuntu servers.
- Configure secure passwordless SSH communication between servers.
- Synchronize server files using rsync over SSH.
- Automate the migration of installed software packages.
- Configure Nginx as a load balancer between Ubuntu servers.
- Produce reusable Bash scripts that can be executed on newly deployed Ubuntu servers.

1.4 Technologies Used

Technology	Purpose
Ubuntu Linux	Operating System
Bash	Automation Scripting
OpenSSH	Secure Remote Connection
rsync	Server Synchronization
Nginx	Load Balancing
SCP	Secure File Transfer
SSH	Remote Server Administration
dpkg	Package export/import
apt	Package installation and management
VirtualBox	Virtual machine environment

1.5 Expected Project Outcome

Upon completion of this project, a system administrator should be able to deploy a new Ubuntu server and configure it using the provided Bash automation scripts instead of manually executing individual commands.

The expected outcome includes:

- Automated server preparation.
- Automated SSH configuration.
- Automated server synchronization.
- Automated package migration.
- Automated Nginx configuration.
- Reduced deployment time.
- Consistent server configurations across multiple Ubuntu servers.

2.1 Environment Requirements

Before implementing the automation scripts, a suitable environment must be prepared to ensure successful communication, synchronization, and migration between the Ubuntu servers. This chapter outlines the hardware, software, networking, and system requirements necessary for implementing the Bash automation solution.

2.2 Hardware Requirements

The implementation was carried out using a standard personal computer capable of running multiple virtual machines simultaneously.

Components	Specification
Processor	Intel Core i5 (or equivalent)
Memory (RAM)	Minimum 8 GB (16 GB Recommended)
Storage	Minimum 100 GB Available Disk Space
Virtualization	Intel VT-x / AMD-V Enabled
Internet Connection	Required

2.3 Software Requirements

Already stated in 1.4 Technologies Used.

2.4 Virtual Machine Configuration

Two Ubuntu virtual machines were created to simulate a production environment.

- Ubuntu01 is the source server.
- Ubuntu02 is the destination server.

Each virtual machine was configured with:

- Ubuntu Server installed

- Static IP Address
- OpenSSH Server installed
- Internet connectivity
- Same local network

2.5 Network Configuration

Both Ubuntu virtual machines were configured to communicate on the same local network. Initially, the servers obtained their IP addresses dynamically using DHCP. However, dynamic IP addresses can change whenever the virtual machine is restarted, which would interrupt SSH communication, file synchronization, package migration, and Nginx load balancing. Assigning static IP addresses ensures that each server always maintains the same network address. This allows the automation scripts to consistently communicate with the correct destination server without requiring IP address updates after every reboot.

To ensure reliable communication between the two Ubuntu servers, static IP addresses were configured using Ubuntu's Netplan configuration.

Server	IP Address	Role
Ubuntu01	192.168.0.70	Source Server
Ubuntu02	192.168.0.71	Destination Server

2.5.1 Netplan Configuration

Ubuntu Server uses Netplan for network configuration.

The following configuration file was modified on both Ubuntu servers:

```
/etc/netplan/50-cloud-init.yaml
```

(Note: Depending on the Ubuntu installation, the filename may differ, such as `/etc/netplan/00-installer-config.yaml`. In this project, use the filename that existed on your virtual machines.)

2.5.2 Example Configuration for Ubuntu01

```
network:

  version: 2

  ethernets:

    enp0s3:

      dhcp4: no

      addresses:

        - 192.168.0.70/24

      gateway4: 192.168.0.1

      nameservers:

        addresses:

          - 8.8.8.8

          - 8.8.4.4
```

2.5.3 Example Configuration for Ubuntu02

```
network:

  version: 2

  ethernets:

    enp0s3:

      dhcp4: no

      addresses:

        - 192.168.0.71/24
```

```
gateway4: 192.168.0.1
```

```
nameservers:
```

```
addresses:
```

```
- 8.8.8.8
```

```
- 8.8.4.4
```

2.5.4 Applying the Configuration

After modifying the Netplan configuration file, the following command was executed on both servers:

```
sudo netplan apply
```

2.5.5 Verifying the Configuration

The following command was used to verify that the static IP address had been successfully assigned:

```
ip addr show
```

The output confirmed that:

- Ubuntu01 was assigned 192.168.0.70
- Ubuntu02 was assigned 192.168.0.71

Network connectivity was then verified by successfully pinging each server from the other.

2.6 Required Ubuntu Packages

Before executing the automation scripts, the following packages must be available.

Package	Purpose
nginx	Load balancer
openssh-server	SSH Server
openssh-client	SSH Client
rsync	File Synchronization

These packages are installed automatically by the `setup.sh` script if they are not already present on the system.

2.7 Connectivity Verification

Before executing any automation scripts, network connectivity should be verified. Verifying connectivity ensures that both Ubuntu servers can communicate successfully before attempting SSH configuration, file synchronization, package migration, or load balancing. Failure to confirm connectivity may result in SSH connection errors or synchronization failures later in the implementation.

To verify connectivity from Ubuntu01:

```
ping 192.168.0.71
```

To verify connectivity from Ubuntu02:

```
ping 192.168.0.70
```

The `ping` command sends Internet Control Message Protocol (ICMP) echo requests to the destination server. Successful replies confirm that both servers are reachable over the network. Connectivity is verified when both servers can successfully ping each other without packet loss. Successful execution should produce replies similar to the following bellow:

```
64 bytes from 192.168.0.71: icmp_seq=1 ttl=64 time=0.xxx ms 64
bytes from 192.168.0.71: icmp_seq=2 ttl=64 time=0.xxx ms
```

2.8 Troubleshooting Encountered

During implementation, the following checks were performed whenever connectivity issues were suspected:

- Confirmed that both virtual machines were connected to the same VirtualBox network.
- Ensure the VirtualBox network is connected to the internet.
- Verified the assigned static IP addresses.
- Confirmed that both servers were powered on.
- Re-ran the `ping` command after correcting any network configuration issues.

3.1 setup.sh (Ubuntu Server Initial Setup Automation)

Before server synchronization can occur, both Ubuntu servers must contain the basic software and services required for communication and data transfer.

Performing this configuration manually on multiple servers is repetitive and time-consuming. There is also a possibility of forgetting important packages or failing to start required services.

The setup.sh script automates the entire initial configuration process by:

- Updating the Ubuntu package repository.
- Installing all required software packages.
- Checking whether packages already exist before attempting installation.
- Enabling and starting essential services.
- Verifying that the services are running successfully.

This guarantees that every Ubuntu server begins from the same configuration baseline before further automation tasks are performed.

3.2 Complete Script

```
#!/bin/bash

echo "===== "
echo "  Ubuntu Server Initial Setup  "
echo "===== "

echo " "

echo "  Updating package list..."

sudo apt update
```

```
echo ""

echo "Checking and Installing required packages..."

packages=(
    openssh-server
    rsync
    nginx
    curl
    git
)

for package in "${packages[@]"; do
    if dpkg -l | grep -q "^ii $package"; then
        echo "$package is already installed."
    else
        echo " Installing $package..."
        sudo apt install -y "$package"
    fi
done

echo ""
```

```
echo "Starting SSH service..."
```

```
sudo systemctl enable ssh
```

```
sudo systemctl start ssh
```

```
echo ""
```

```
echo "Starting Nginx..."
```

```
sudo systemctl enable nginx
```

```
sudo systemctl start nginx
```

```
echo ""
```

```
echo "Checking installed services..."
```

```
systemctl is-active ssh
```

```
systemctl is-active nginx
```

```
echo ""
```

```
echo "=====
```

```
echo "Setup completed successfully."
```

```
echo "=====
```

3.3 Explanation of Every Command

The `setup.sh` script automates the initial configuration of an Ubuntu server by performing the following tasks:

1. Updating the Package Repository

```
sudo apt update
```

- Updates the local package index from the Ubuntu repositories.
- Ensures that the latest versions of software packages are available for installation.

2. Creating the Required Package List

```
packages=(  
    openssh-server  
  
    rsync  
  
    nginx  
  
    curl  
  
    git  
)
```

- Creates an array containing all software required for the project.
- Using an array makes it easy to add or remove packages in the future without modifying multiple parts of the script.

3. Looping Through Each Package

```
for package in "${packages[@]}"; do
```

- Iterates through every package stored in the array.
- Eliminates the need to write individual installation commands for each package.

4. Checking Whether a Package Already Exists

```
if dpkg -l | grep -q "^ii $package"; then
```

- Determines whether the package is already installed.
- Prevents unnecessary reinstallation of software.

5. Installing Missing Packages

```
sudo apt install -y "$package"
```

- Installs only packages that are not already present.
- The `-y` option automatically accepts installation prompts, allowing the script to run without user intervention.

6. Enabling and Starting the SSH Service

```
sudo systemctl enable ssh
```

```
sudo systemctl start ssh
```

- Configures the SSH service to start automatically during system boot.
- Starts the SSH service immediately so that remote administration is available.

7. Enabling and Starting the Nginx Service

```
sudo systemctl enable nginx
```

```
sudo systemctl start nginx
```

- Ensures the Nginx web server starts automatically whenever the server boots.
- Starts the web server immediately after installation.

8. Verifying Service Status

```
systemctl is-active ssh
```

```
systemctl is-active nginx
```

- Confirms that both SSH and Nginx services are running correctly.
- Displays active if the services have started successfully.

9. Displaying Completion Message

```
echo "Setup completed successfully."
```

- Displays a confirmation message indicating that all setup operations completed without errors.

3.4 Script Execution

After creating the script and saving it as `setup.sh`, the following steps were performed to execute it.

Step 1: Grant Execute Permission

Before a Bash script can be executed, it must be given executable permissions.

```
chmod +x setup.sh
```

- `chmod` changes file permissions.
- `+x` adds execute permission to the script.
- `setup.sh` is the name of the Bash script.

Step 2: Execute the Script

Run the script from its current directory.

```
./setup.sh
```

4.1 configure_ssh.sh

The `configure_ssh.sh` script automates the configuration of Secure Shell (SSH) authentication between the source Ubuntu server and the destination Ubuntu server.

Normally, an administrator would have to manually generate SSH keys, copy the public key to the remote server, verify authentication, and repeatedly enter passwords whenever connecting to the destination server. This process is time-consuming and inefficient, especially when subsequent automation scripts depend on passwordless SSH communication.

This script eliminates the manual process by automatically:

- Checking whether an SSH key pair already exists.
- Generating a new SSH key pair if one does not exist.
- Copying the SSH public key to the destination server.
- Testing passwordless SSH authentication.
- Confirming successful SSH connectivity.

This script is essential because both the server synchronization script (`sync_server.sh`) and the package migration script (`install_packages.sh`) rely on secure SSH communication between the two Ubuntu servers.

4.2 Complete Script

```
#!/bin/bash
```

```
echo "=====
```

```
echo "  SSH Configuration Script  "
```

```
echo "=====
```

```
DEST_USER=$1
```

```
DEST_IP=$2
```

```
if [ -z "$DEST_USER" ] || [ -z "$DEST_IP" ]; then

    echo "ERROR: Missing arguments. Usage: ./configure_ssh.sh
<username> <ip>"

    exit 1

fi

echo ""

echo "Checking for an existing SSH key..."

if [ -f ~/.ssh/id_rsa ]; then

    echo "SSH key already exists."

else

    echo "Generating a new SSH key..."

    ssh-keygen -t rsa -b 4096 -N "" -f ~/.ssh/id_rsa

fi

echo ""

echo "Copying SSH key to $DEST_USER@$DEST_IP ..."

ssh-copy-id "$DEST_USER@$DEST_IP"

echo ""
```

```
echo "Testing SSH connection..."
```

```
ssh -o BatchMode=yes "$DEST_USER@$DEST_IP" "echo SSH connection  
successful."
```

```
echo " "
```

```
echo "=====
```

```
echo "SSH configuration completed."
```

```
echo "=====
```

4.2 Explanation of the Script

1. if [-f ~/.ssh/id_rsa]

Checks whether an SSH private key already exists.

```
if [ -f ~/.ssh/id_rsa ]
```

If the file exists:

SSH key already exists.

is displayed.

Otherwise, the script generates a new key pair.

2. ssh-keygen

Creates a new RSA SSH key pair.

```
ssh-keygen -t rsa -b 4096 -N "" -f ~/.ssh/id_rsa
```

Explanation

Command	Function
-t rsa	Generates an RSA key pair
-b 4096	Uses a 4096-bit encryption key
-N""	Creates the key without a passphrase
-f ~/.ssh/id_rsa	Saves the key pair in the default SSH directory

3. ssh-copy-id

Copies the public SSH key to the destination server.

```
ssh-copy-id "$DEST_USER@$DEST_IP"
```

During the first execution, the administrator is prompted to enter the destination server password once.

After the key is copied successfully, future SSH logins no longer require password authentication.

4. ssh -o BatchMode=yes

Tests passwordless SSH authentication.

```
ssh -o BatchMode=yes "$DEST_USER@$DEST_IP" "echo SSH connection  
successful."
```

BatchMode=yes **disables** password prompts.

If passwordless authentication has been configured successfully, the command returns:

SSH connection successful.

Otherwise, the connection fails, indicating that SSH key authentication has not been configured correctly.

3.4 Script Execution

After creating the script and saving it as `configure_ssh.sh`, the following steps were performed to execute it.

Step 1: Grant Execute Permission

Before a Bash script can be executed, it must be given executable permissions.

```
chmod +x configure_ssh.sh
```

- `chmod` changes file permissions.
- `+x` adds execute permission to the script.
- `configure_ssh.sh` is the name of the Bash script.

Step 2: Execute the Script

Run the script from its current directory.

```
./configure_ssh.sh
```

4.1 sync_server.sh

The `sync_server.sh` script automates the synchronization of the Ubuntu source server with the destination Ubuntu server using the `rsync` utility over SSH.

During server migration, manually copying files and directories is time-consuming and increases the likelihood of missing important system files or changing file permissions. This script ensures that the destination server becomes an exact replica of the source server by securely transferring files while preserving permissions, ownership, Access Control Lists (ACLs), extended attributes, symbolic links, hard links, and timestamps.

To prevent system instability, runtime directories and virtual file systems such as `/proc`, `/sys`, `/dev`, and `/run` are excluded because they are recreated automatically by Linux during boot.

This automation significantly reduces migration time while ensuring consistency and integrity of the migrated server.

4.2 Complete Script

```
#!/bin/bash

echo
"===== "

echo "Ubuntu Server Synchronization"

echo
"===== "

DEST_USER=$1

DEST_IP=$2

if [ -z "$DEST_USER" ] || [ -z "$DEST_IP" ]; then

    echo "ERROR: Missing arguments. Usage: ./sync_server.sh  
<username> <ip>"

    exit 1

fi

echo " "

echo "Starting synchronization..."

echo " "
```

```
sudo rsync -aAXHv \  
--delete \  
--rsync-path="sudo rsync" \  
--exclude=/proc \  
--exclude=/sys \  
--exclude=/dev \  
--exclude=/run \  
--exclude=/tmp \  
--exclude=/media \  
--exclude=/mnt \  
--exclude=/lost+found \  
--exclude=/swap.img \  
-e ssh \  
/ \  
${DEST_USER}@${DEST_IP}:/\  
  
if [ $? -eq 0 ]; then  
echo "  
echo "===== "  
echo "Synchronization Complete"
```

```
echo "===== "  
  
else  
  
echo " "  
  
echo "===== "  
  
echo "Synchronization Failed."  
  
echo "===== "  
  
Fi
```

5.1 install_packages.sh

The `install_packages.sh` script automates the migration of installed software packages from the source Ubuntu server to the destination Ubuntu server.

During server migration, manually identifying and reinstalling every installed application is time-consuming and prone to errors. This script automatically exports the list of installed packages from the source server, securely transfers the package list to the destination server, and installs the same packages on the destination system.

Automating this process ensures that both servers have a consistent software environment while significantly reducing the time required to rebuild the destination server.

5.2 Complete Script

```
#!/bin/bash
```

```
echo "===== "  
echo "Ubuntu Server Package Synchronization"  
echo "===== "  
  
read -p "Destination username: " DEST_USER  
read -p "Destination IP: " DEST_IP  
  
echo ""  
echo "Exporting installed packages..."  
  
dpkg --get-selections > installed_packages.txt  
echo "Package list exported."  
  
echo ""  
echo "Copying package list to destination server..."  
  
scp "$HOME/installed_packages.txt" \  
"$DEST_USER@$DEST_IP:/tmp/"  
  
echo ""
```

```

echo "Installing packages on destination server..."

ssh -t "$DEST_USER@$DEST_IP" << EOF

sudo apt-get update

sudo dpkg --set-selections < /tmp/installed_packages.txt

sudo apt-get dselect-upgrade -y

EOF

if [ $? -eq 0 ]; then

    echo " "

    echo "===== "

    echo "Package Synchronization Complete"

    echo "===== "

Fi

```

6.1 Configuration-Driven Architecture

To achieve a "Zero-Touch" deployment model, the Ubuntu automation was upgraded from interactive scripts (which required manual input via `read -p`) to a configuration-driven architecture. This allows an administrator to deploy and configure a new server simply by editing a configuration file and running a single master script.

The environment now utilizes an environment configuration file named `server_config.env`. This file stores all critical variables, including server roles, IP addresses, SSH usernames, and Cron schedules.

`server_config.env` Structure:

```
ROLE="PRIMARY"
```

```
PRIMARY_IP="192.168.0.70"
```

```
SECONDARY_IP="192.168.0.71"
```

```
SSH_USER="sysadmin"
```

```
SYNC_SCHEDULE="0 2 * * 0"
```

6.2 The Master Orchestrator Script (master_install.sh)

The `master_install.sh` script acts as the central orchestrator. It sources the `server_config.env` file and sequentially executes the sub-scripts (`setup.sh`, `configure_ssh.sh`, `install_packages.sh`, and `sync_server.sh`).

The sub-scripts were updated to accept Command-Line Arguments (\$1 for username, \$2 for IP) instead of using `read -p`. The Master Script automatically passes the variables from the config file to these sub-scripts, allowing the entire installation to run unattended.

6.3 Automated Synchronization via Cron

Previously, synchronization had to be triggered manually. The Master Orchestrator now automatically registers a Cron job upon completion.

The script programmatically edits the root user's crontab (`sudo crontab -l`) to append the synchronization command using the schedule defined in `SYNC_SCHEDULE`. This ensures that Ubuntu02 remains consistently synchronized with Ubuntu01 via `rsync` over SSH every week at the specified time, completely unattended.

6.4 Complete Script (master_install.sh)

```
#!/bin/bash
```

```
# master_install.sh
```

```
# 1. Load Configuration
```

```
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
```

```
source "$SCRIPT_DIR/server_config.env"
```

```
echo "===== "
echo " Unattended Ubuntu Master Installation"
echo " Role: $ROLE | Peer IP: $SECONDARY_IP"
echo "===== "

# 2. Run Initial Setup (Packages, Nginx, SSH services)
echo ""
echo "[1/4] Running Initial Ubuntu Setup..."
bash "$SCRIPT_DIR/setup.sh"

# 3. Run SSH Configuration (Passes User and IP automatically)
echo ""
echo "[2/4] Configuring Passwordless SSH..."
bash "$SCRIPT_DIR/configure_ssh.sh" "$SSH_USER" "$SECONDARY_IP"

# 4. Run Package Migration
echo ""
echo "[3/4] Migrating Installed Packages..."
bash "$SCRIPT_DIR/install_packages.sh" "$SSH_USER"
"$SECONDARY_IP"

# 5. Run Initial Sync & Setup Automated Cron Schedule
echo ""
echo "[4/4] Running Initial Sync and Registering Cron
Schedule..."

# Run the sync script once immediately
bash "$SCRIPT_DIR/sync_server.sh" "$SSH_USER" "$SECONDARY_IP"

# Setup Cron Job for automated syncing
# We use sudo crontab because sync_server.sh uses 'sudo rsync'
CRON_JOB="$SYNC_SCHEDULE sudo bash $SCRIPT_DIR/sync_server.sh
$SSH_USER $SECONDARY_IP >> /var/log/server_sync.log 2>&1"
(sudo crontab -l 2>/dev/null | grep -v
"$SCRIPT_DIR/sync_server.sh"; echo "$CRON_JOB") | sudo crontab -
```

```
echo " "  
echo "===== "  
echo " MASTER INSTALLATION COMPLETE! "  
echo " Sync scheduled via Cron: $SYNC_SCHEDULE "  
echo "===== "
```


